MoonWalker Weekly Meeting

11/5 - 11/12 Summary

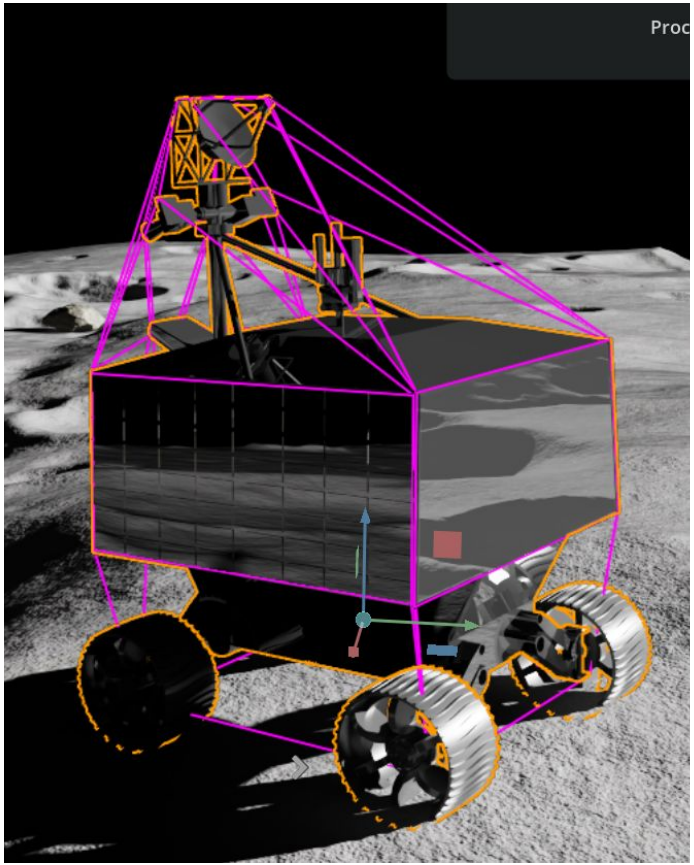
Elijah, Rowel, Martin

Progress Update

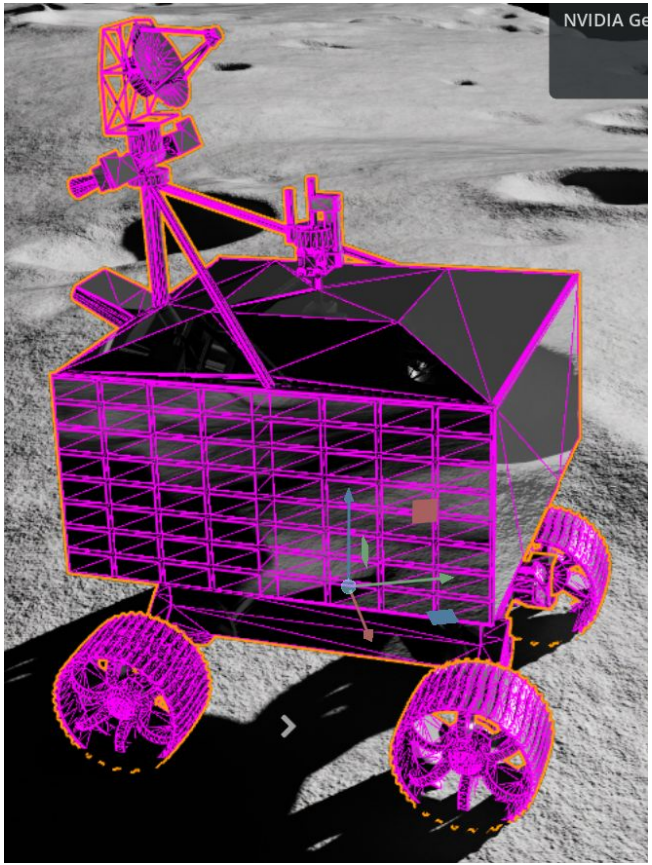
- Rowel
  - Update the API(locally) with is inControl property so that Unity can be designated as the Host
  - Continue research on how to make collisions identical (Shared)
- Elijah
  - Fixed inconsistencies with Collider property in Omniverse
  - Writing Tracker.py to track collisions on the Omniverse side
  - Write the OmniConsumerSwitch.py
  - Continue research on how to make collisions identical (Shared)
- Martin
  - Continuing Tracker.cs to track collisions on the Unity side
  - Update and write OmniProducerSwitch.cs
  - Continue research on how to make collisions identical (Shared)

```
(venv) rowele@Rowels-MacBook-Air-10 src % python3 inControl_test.py

  ======================================================================
  INCONTROL ENFORCEMENT — LOCAL LOGIC TEST
  ======================================================================
  /Users/rowele/hsml_api/src/inControl_test.py:85: DeprecationWarning: datetim
  are objects to represent datetimes in UTC: datetime.datetime.now(datetime.UT
    ts = datetime.utcnow().isoformat() + "Z"
  [2025-11-12T09:15:04.410511Z] CASE: Host message (should be accepted)
  → POST /producer/send-message?topic=feedback_channel
    inControl : True
  [producer/send-message] topic=feedback_channel, inControl=True
  [DummyProducer] produce()
  [DummyProducer] flush()
  ← HTTP 200 | expected 200 → PASS ✅
    body     : {"message":"Message sent successfully"}
  [2025-11-12T09:15:04.437581Z] CASE: Non-host message (should be blocked)
  → POST /producer/send-message?topic=feedback_channel
    inControl : False
  ← HTTP 409 | expected 409 → PASS ✅
    body     : {"detail":"not-host: inControl is False"}

  Summary: if you saw 200 then 409, `inControl` gate is working.

(venv) rowele@Rowels-MacBook-Air-10 src %
```

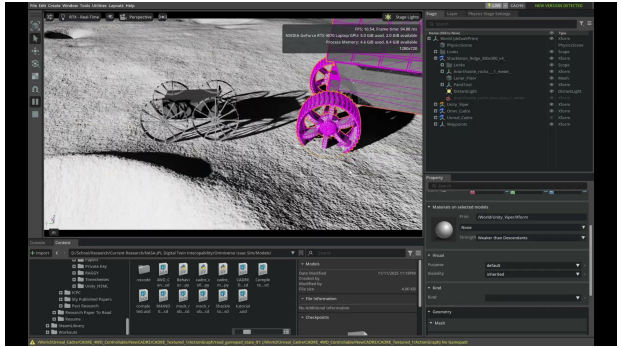
inControl_test script

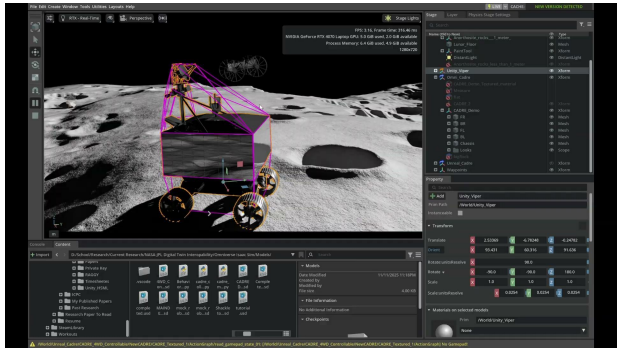


Convex Hull

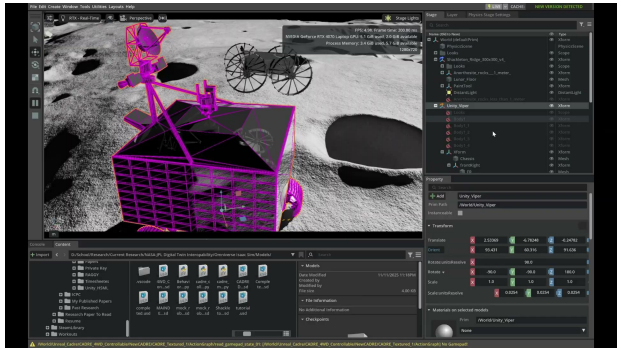


Triangle Mesh





Convex Hull



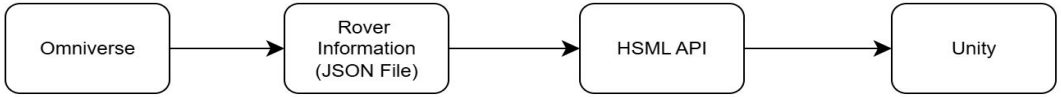
Triangle Mesh

File Edit Create Window Tools Utilities Layouts Help
LIVE
CACHE
NEW VERSION DETECTED
RTX - Real-Time
Perspective
Stage Lights
FPS: 12.60, Frame time: 79.37 ms
NVIDIA GeForce RTX 4070 Laptop GPU: 5.0 GiB used, 2.0 GiB available
Process Memory: 6.0 GiB used, 6.3 GiB available
1280x720
Stage
Layer
Physics Stage Settings
Search
Name
Type
Lunar_Floor
Mesh
ParentTool
Xform
DistantLight
DistantLight
Unity_Viper
Xform
Omni_Cadre
Xform
CADRE_Demo
Xform
FR
Mesh
BR
Mesh
FL
Mesh
BL
Mesh
Chassis
Mesh
Looks
Scope
Unreal_Cadre
Xform
Waypoints
Xform
Property
Add
CADRE_Demo
Prim Path
/World/Omni_Cadre/CADRE_Demo
Instanceable
Transform
Translate
147.19187
10.99593
-1132.91989
Orient
180.0
-79.809
180.0
Scale
1.0
1.0
1.0
Rotate
0.0
0.0
0.0
Materials on selected models
Prim
/World/Omni_Cadre/CADRE_Demo
None
Strength
Weaker than Descendants
Console
Content
Import
D:/School/Research/Current Research/NASA-JPL Digital Twin Interoperability/Omniverse Isaac Sim/Models/
Papers
Private Key
KAGGY
Timesheetes
Unity_HSML
KPC
My Published Papers
Past Research
Research Paper To Read
Resume
SteamLibrary
Workouts
.vscode
4WD_C
om..sd
Behavi
or .py
cadre.r
oS..py
cadre.
m..py
CADRE
D...sd
Comple
te...sd
comple
ted.usd
MAIND
E...sd
mock_r
oS..sd
mock_r
oS..sd
Shackle
to...sd
tutorial
.usd
Models
Date Modified
11/11/2025 11:18PM
Created by
Modified by
File size
4.00 KB
File Information
No Additional Information
Checkpoints
/World/Unreal_Cadre/CADRE_4WD_Controllable/NewCADRE/CADRE_Textured_1/ActionGraph/read_gamepad_state_01: [/World/Unreal_Cadre/CADRE_4WD_Controllable/NewCADRE/CADRE_Textured_1/ActionGraph] No Gamepad!

Proposed Idea

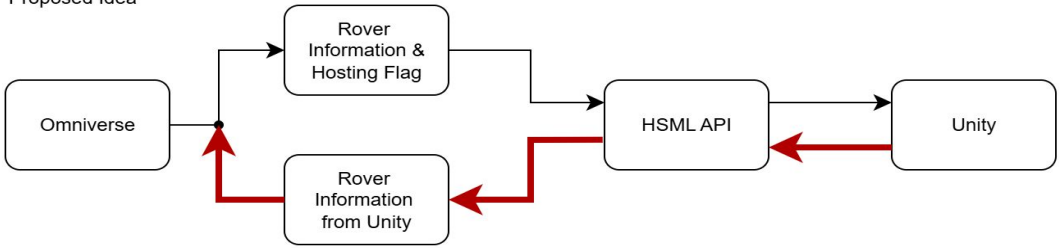
Unity as Host

Previous setup



1. Cadre is controlled by Omniverse
2. Cadre location tracked by Tracker.py
3. JSON file is created with information from Tracker.py and sent to HSML API
4. Unity reads Cadre information and updates
5. Vice versa, Omniverse reads information for Viper from Unity
6. Collision happens, the respective simulation platform handles physics separately

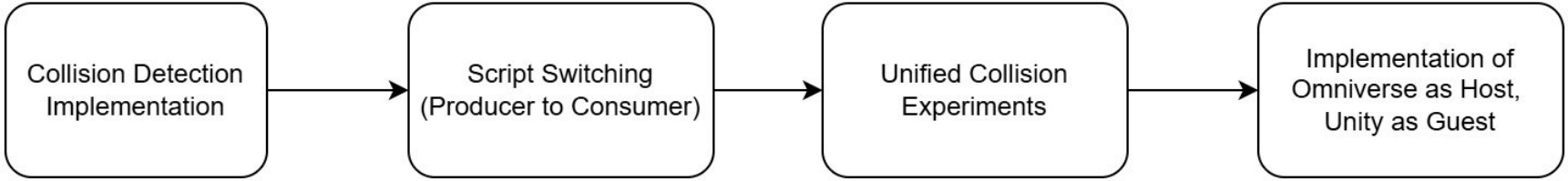
Proposed Idea



1. Omniverse set as guest, Unity set as host
2. Cadre location tracked by Tracker.py
    a. **Tracker.py configured to also detect for collisions**
3. JSON file is created with information from Tracker.py and sent to HSML API
    a. **Hosting flag and collision information is also indicated in JSON file. Hosting = 0 if platform is guest**
4. If no collision, Unity reads Cadre information and updates
5. **If collision:**
    a. **Unity takes over physics for cadre**
    b. **Omniverse turns off cadre movement script and switches to consumer (Consumer Script for Cadre in Omniverse needed)**
    c. **Omniverse stops producing rover information**

Implementation Plan

Proposed Idea Implementation: Unity as Host, Omniverse as Guest



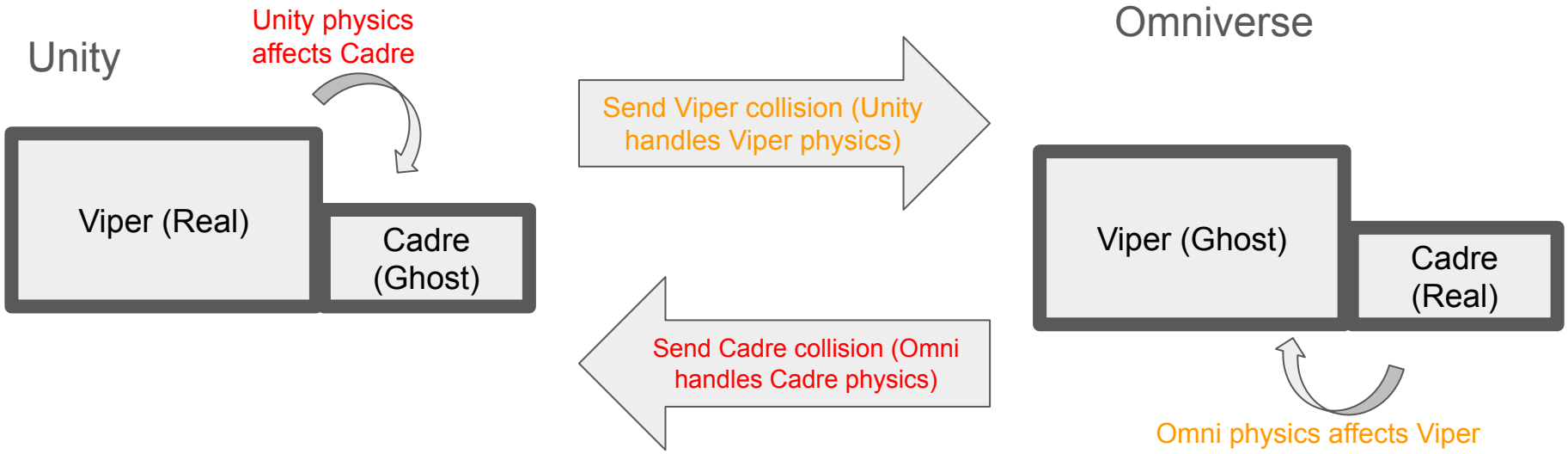
Items Required:
1. Collision detection scripts for the individual simulation platforms

Items Required:
1. JSON producer file updated to contain hosting flag
2. Consumer and Producer scripts for each rover. Total of 4 scripts per simulation platform.
3. Implementing switching of producer/consumer scripts in collision detection scripts

Items Required:
1. Experiment documentation for setup
2. Experiment setup definition

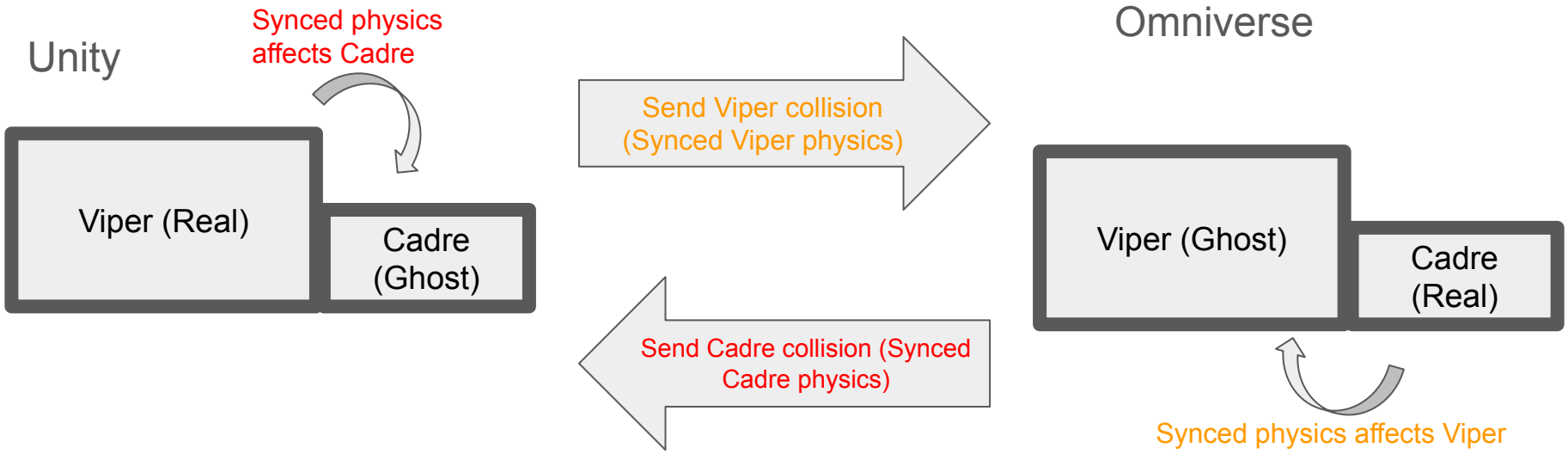
Backup Idea

- In both Unity and Omniverse, “real” and “ghost” rovers must have a collider. If not, no collision will never occur.
  - Due to this notion, we <u>hypothesize</u> that local physics interferes with consumer data since both engines need to process the interaction.
- Note visual below: Each ghost rover is affected by two different physics.



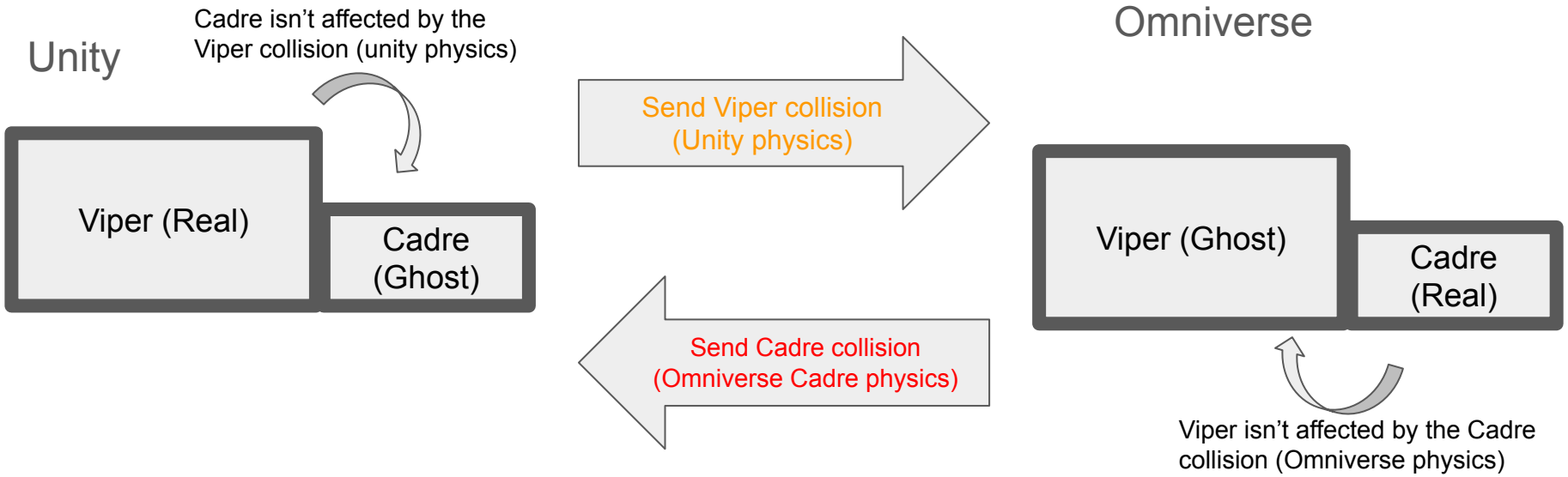
Backup Idea

- Simply finding a way to make both simulations use the same metrics to process local physics **may** still not work. (would need thorough testing)
- Note visual: Each ghost rover is affected by the same physics but twice. Unknown if any weird unrealistic interaction could occur such as collisions effects are double.



Backup Idea

- <u>Proposal</u>: The idea is that we need each “real” rover in each simulation to be processed by their respective local physics while each “ghost” rover must **not** be affected by their respective local physics.
  - Pros: No need to designate one platform as host
  - Things to consider: Two different physics are at play; still testing methods on how to ignore local physics (currently looking into applying kinematic properties, unsure if possible yet)



Thoughts and Questions

- As we continue with our implementation of a hosting platform how can we not stray away from digital twin interoperability?
- Is asynchronous simulations something that will be worked toward in the future because in the real world is it realistic for users to be online at the same time.

Action Plan

- Rowel
  - Migrate the updated code to the DT Server
  - Test API with implemented inControl property during Collision testing
    - Update the code so it does not impact existing scripts too much
  - Learn more about Physics engine difference between Unity and Omniverse
- Elijah
  - Learn more about Physics engine difference between Unity and Omniverse
  - Finish up collision detection script for Omniverse
  - Create Producer/Consumer switching script for Omniverse for proposed idea
  - Conduct tests for implementing backup idea
- Martin
  - Update Viper & Cadre Colliders to match Elijah
    - Looking into a way to standardize Colliders boundaries on rovers when importing projects
  - Assist Elijah & Rowel w/ Proposed Idea